Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

INTRODUCTION

In today's digital world, organizations depend heavily on computer systems, networks, and cloud applications to store and manage important data. As the use of digital technologies increases, the risk of cyber threats and unauthorized access also grows. Traditional access control methods that rely only on usernames and passwords are no longer strong enough to protect sensitive systems. Therefore, modern security solutions such as context-aware access control are used to improve system security and ensure that only authorized users can access important resources.

Context-aware access control is a security approach that grants or restricts access based on different contextual factors instead of relying only on basic authentication. These factors include user identity, device posture, location, and application risk. User identity verification ensures that the person requesting access is legitimate by using methods such as strong passwords or multi-factor authentication. Device posture checks whether the device used for access meets security requirements like updated software, antivirus protection, and proper security configurations. Location awareness helps identify whether the access request is coming from a trusted network or geographical location.

In addition, context-aware access control also evaluates the risk level of the application being accessed. Applications that handle sensitive or confidential data require stronger authentication and stricter access policies. By combining identity verification, device security checks, location monitoring, and application risk evaluation, organizations can create a more intelligent and dynamic security system. This approach supports modern security frameworks such as the Zero Trust model, where every access request is continuously verified to reduce the risk of unauthorized access and protect sensitive information.

ABSTRACT

Context-aware access control is an advanced security approach designed to improve the protection of systems, networks, and applications by considering multiple contextual factors before granting access. Traditional security methods mainly rely on usernames and passwords, which are often insufficient to prevent modern cyber threats. In contrast, context-aware access control evaluates factors such as **user identity, device posture, location, and application risk** to determine whether a user should be allowed to access a particular resource. This approach helps organizations make more accurate and secure access decisions.

By analyzing these contextual elements, organizations can ensure that only trusted users using secure devices from authorized locations can access sensitive applications. Additionally, high-risk applications can be protected with stronger authentication and stricter access policies. This method supports modern security models such as **Zero Trust Architecture**, where every access request is continuously verified. As a result, context-aware access control helps reduce unauthorized access, minimize security risks, and strengthen overall system security in modern digital environments.

USE CASE SCENARIOS

1. Secure Employee Login from Trusted Device

Scenario:
An employee attempts to access the company's internal portal using their registered laptop from the office network. The system checks the user's identity through login credentials and verifies the device posture, ensuring that the device has updated security patches and antivirus protection.

Output:
Access is granted because the user identity is valid, the device is compliant with security policies, and the login attempt is from a trusted location.

2. Access Attempt from Untrusted Location

Scenario:
A user tries to log into the organization's system from a different country that is not normally associated with the user's activity. The context-aware access control system detects the unusual location.

Output:
The system triggers **additional verification such as multi-factor authentication (MFA)** or temporarily blocks access until the user identity is confirmed.

3. Login from Non-Compliant Device

Scenario:
A user attempts to access the system using a device that does not meet security requirements, such as missing antivirus protection or outdated operating system updates.

Output:
The system denies access and prompts the user to update the device security settings before allowing login.

4. Access to High-Risk Application

Scenario:

A user attempts to access a financial application that contains sensitive organizational data. Since the application is classified as high-risk, the system applies stricter authentication checks.

Output:

The system requires **multi-factor authentication (MFA)** and device verification before granting access.

5. Suspicious Login Behavior Detected

Scenario:

A user logs in from a trusted device but suddenly attempts to access multiple sensitive applications within a short period of time, which is unusual compared to normal behavior.

Output:

The system flags the activity as suspicious, temporarily restricts access, and alerts the security administrator for further investigation.

Stage-1

Title of the Project

Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Overview

Design and apply context-aware access controls based on user identity, device posture, location, and application risk is a modern security approach used to protect systems and sensitive data. Traditional access control methods mainly rely on usernames and passwords, which are no longer sufficient to prevent modern cyber threats. Context-aware access control improves security by evaluating multiple factors before granting access to a system or application.

In this approach, **user identity** is verified through authentication methods such as passwords, biometrics, or multi-factor authentication (MFA). This ensures that only authorized users can request access. Another important factor is **device posture**, which checks whether the device being used meets security requirements such as updated operating systems, antivirus protection, and security patches.

Location is also considered when evaluating access requests. For example, an organization may allow access only from trusted networks or specific geographic locations. If a login attempt occurs from an unusual location, additional verification may be required. Additionally, the system evaluates the **risk level of the application** being accessed. Applications that handle sensitive data require stronger security controls compared to low-risk applications.

By combining these contextual factors, organizations can create a **dynamic and intelligent access control system** that adapts to different situations. This approach improves security, reduces unauthorized access, and supports modern security frameworks such as the **Zero Trust model**.

LIST OF TEAMMATES:

S.no	Name	College	Contact
1.	Varshini M – Team leader	AVP College of Arts and Science	varshinimuthukumar13@gmail.com
2.	Vinisha R	AVP College of Arts and Science	vinisharajendran393@gmail.com
3.	Ninitha R	AVP College of Arts and Science	ninitharajkumar03@gmail.com
4.	Nanthini P	AVP College of Arts and Science	nanthinipalanivel24@gmail.com
5.	Sandhiya G	AVP College of Arts and Science	sandhiyaganeshan22@gmail.com

LIST OF VULNERABILITIES

S.no	Vulnerability Names	CWE/CVE - No
1.	Use of Hard-coded Credentials	CWE:798 – Use of Hard-coded Credentials
2.	Channel Accessible by Non-Endpoint (Man-in-the-Middle)	CWE:300 – Channel Accessible by Non-Endpoint
3.	Session Fixation	CWE:384 – Session Fixation
4.	Untrusted Search Path	CWE: 426 – Untrusted Search Path
5.	Embedded Malicious Code	CWE: 506 – Embedded Malicious Code
6.	Download of Code Without Integrity Check	CWE: 494 – Download of Code Without Integrity Check
7.	Weak Multi-Factor Authentication Enforcement	CWE: 308 – Use of Single Factor Authentication
8.	Weak Encoding for Password	CWE: 261 – Weak Encoding for Password
9.	Missing or Improper Software Bill of Materials (SBOM) Validation	CWE: 1104 – Use of Unmaintained Third-Party Components
10.	Missing Cryptographic Step	CWE: 325 – Missing Cryptographic Step

REPORTS

1.Vulnerability Name: Use of Hard-coded Credentials

CWE: 798 – Use of Hard-coded Credentials

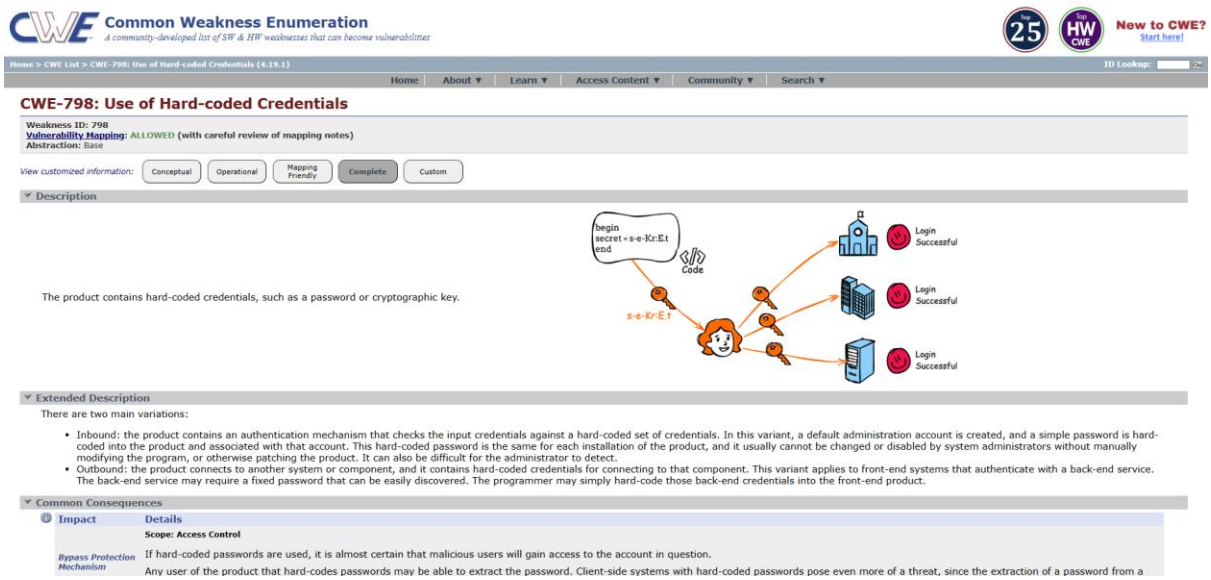
OWASP Category: A02:2021 – Cryptographic Failures

Description:

This vulnerability occurs when credentials such as usernames, passwords, or cryptographic keys are hard-coded directly into the application's source code. Since these credentials are fixed and cannot be easily changed, attackers who access the code can discover and misuse them.

Business Impact:

Attackers can extract the hard-coded credentials and gain unauthorized access to systems, databases, or services. This may lead to data breaches, privilege escalation, and compromise of sensitive information.



Vulnerability Parameter:

Hard-coded username or password stored in application code.

Steps to Reproduce:

- Review the application source code or configuration files.
- Identify credentials embedded directly in the code.
- Use the discovered credentials to authenticate and access the system.

Recommendation:

- Avoid storing credentials directly in source code.
- Use secure credential storage such as environment variables or secret management tools.
- Implement strong authentication and regularly rotate credentials.

2.Vulnerability Name: Channel Accessible by Non-Endpoint (Man-in-the-Middle)

CWE: 300 – Channel Accessible by Non-Endpoint

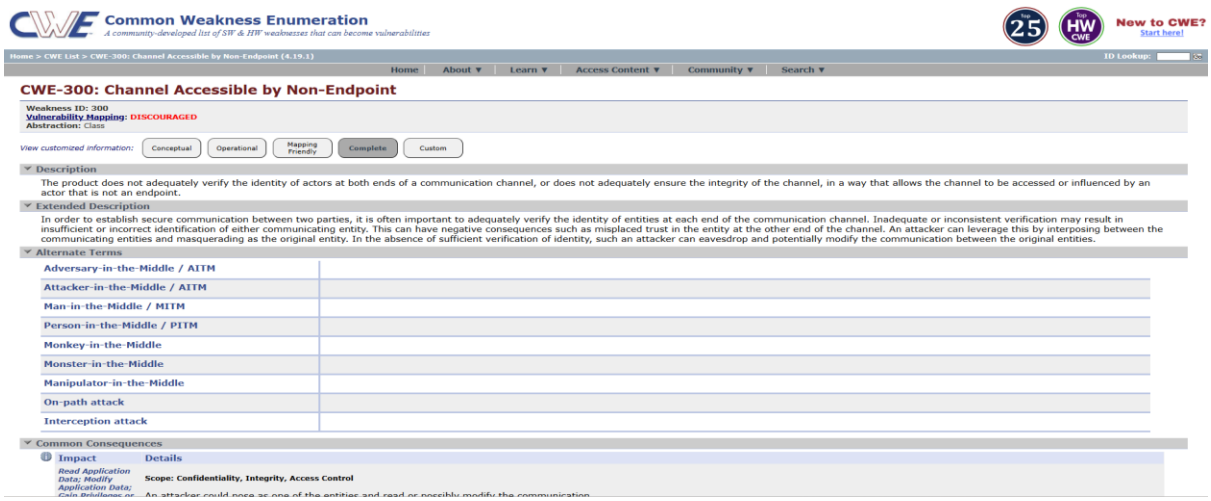
OWASP Category: A02:2021 – Cryptographic Failures

Description:

This vulnerability occurs when a communication channel between two systems does not properly verify the identity of both endpoints. An attacker can intercept the communication and position themselves between the sender and receiver, allowing them to read or modify the transmitted data.

Business Impact:

Attackers can eavesdrop on sensitive communications, steal confidential information, modify transmitted data, or impersonate legitimate users. This may lead to data breaches, loss of integrity, and unauthorized access.



Vulnerability Parameter:

Unsecured communication channel between client and server.

Steps to Reproduce:

- Identify communication between client and server over an unsecured channel.
- Intercept the traffic using a network interception tool.
- Observe that the attacker can view or modify the transmitted data.

Recommendation:

- Use secure communication protocols such as HTTPS/TLS.
- Implement strong certificate validation and authentication mechanisms.
- Ensure proper encryption and integrity protection for all sensitive communications.

3.Vulnerability Name: Session Fixation

CWE: 384 – Session Fixation

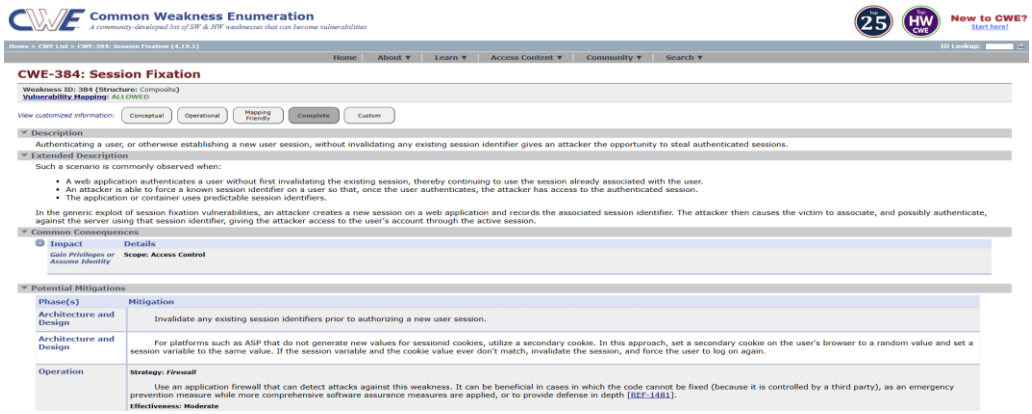
OWASP Category: A07:2021 – Identification and Authentication Failures

Description:

Session fixation occurs when a web application allows a user to authenticate without generating a new session ID. An attacker can set or predict a session ID and trick the victim into using it. Once the victim logs in, the attacker can use the same session ID to access the authenticated session.

Business Impact:

Attackers can hijack user sessions and gain unauthorized access to user accounts. This may lead to identity theft, unauthorized actions, and exposure of sensitive information.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-384: Session Fixation (4.19.1)

ID Lookup:

Home | About ▾ | Learn ▾ | Access Content ▾ | Community ▾ | Search ▾

CWE-384: Session Fixation

Weakness ID: 384 (Structure: Composite)
Vulnerability Mapping: ALLOWED

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier gives an attacker the opportunity to steal authenticated sessions.

Extended Description

Such a scenario is commonly observed when:

- A web application authenticates a user without first invalidating the existing session, thereby continuing to use the session already associated with the user.
- An attacker is able to force a known session identifier on a user so that, once the user authenticates, the attacker has access to the authenticated session.
- The application or container uses predictable session identifiers.

In the generic exploit of session fixation vulnerabilities, an attacker creates a new session on a web application and records the associated session identifier. The attacker then causes the victim to associate, and possibly authenticate, against the server using that session identifier, giving the attacker access to the user's account through the active session.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	**Scope: Access Control**

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	Invalidate any existing session identifiers prior to authorizing a new user session.
Architecture and Design	For platforms such as ASP that do not generate new values for sessionid cookies, utilize a secondary cookie. In this approach, set a secondary cookie on the user's browser to a random value and set a session variable to the same value. If the session variable and the cookie value ever don't match, invalidate the session, and force the user to log on again.
Operation	**Strategy: *Firewall*** Use an application firewall that can detect attacks against this weakness. It can be beneficial in cases in which the code cannot be fixed (because it is controlled by a third party), as an emergency prevention measure while more comprehensive software assurance measures are applied, or to provide defense in depth [REF-1481]. **Effectiveness: Moderate**

Vulnerability Parameter:

Session identifier used in cookies or URL parameters.

Steps to Reproduce:

- Create or obtain a valid session ID from the application.
- Send the session ID to the victim through a crafted link.
- The victim logs in using the same session ID.
- The attacker uses that session ID to access the authenticated session.

Recommendation:

- Generate a new session ID after user authentication.
- Invalidate old session identifiers before creating a new session.
- Use secure session management practices and enable secure cookie attributes.

4.Vulnerability Name: Untrusted Search Path

CWE: 426 – Untrusted Search Path

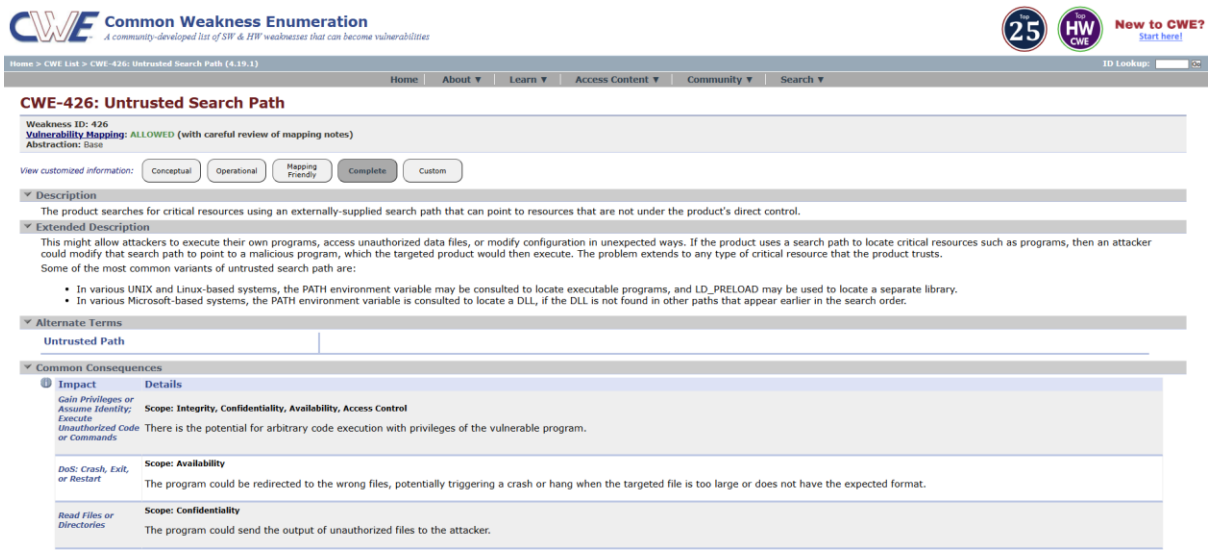
OWASP Category: A05:2021 – Security Misconfiguration

Description:

This vulnerability occurs when an application searches for executable programs or resources using a search path that includes directories not controlled by the application. Attackers can place malicious files in these locations, causing the application to execute the attacker's code instead of the intended program.

Business Impact:

Attackers may execute malicious programs with the privileges of the vulnerable application. This can lead to unauthorized access, data theft, system compromise, or service disruption.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-426: Untrusted Search Path (4.19.1)

ID Lookup:

Home | About ▾ | Learn ▾ | Access Content ▾ | Community ▾ | Search ▾

CWE-426: Untrusted Search Path

Weakness ID: 426
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

Alternate Terms

Untrusted Path

Common Consequences

Impact	Details
Gain Privileges or Assume Identity; Execute Unauthorized Code or Commands	Scope: Integrity, Confidentiality, Availability, Access Control There is the potential for arbitrary code execution with privileges of the vulnerable program.
DoS: Crash, Exit, or Restart	Scope: Availability The program could be redirected to the wrong files, potentially triggering a crash or hang when the targeted file is too large or does not have the expected format.
Read Files or Directories	Scope: Confidentiality The program could send the output of unauthorized files to the attacker.

Vulnerability Parameter:

External search path or environment variables used to locate executable files (e.g., PATH).

Steps to Reproduce:

- Identify the application's search path used to locate executables or libraries.
- Place a malicious executable file in a directory included in the search path.
- Execute the application and observe that the malicious file is executed instead of the legitimate program.

Recommendation:

- Use absolute paths for executing programs and accessing critical resources.
- Restrict search paths to trusted directories only.
- Validate and sanitize environment variables such as PATH before execution.

5.Vulnerability Name: Embedded Malicious Code

CWE: 506 – Embedded Malicious Code

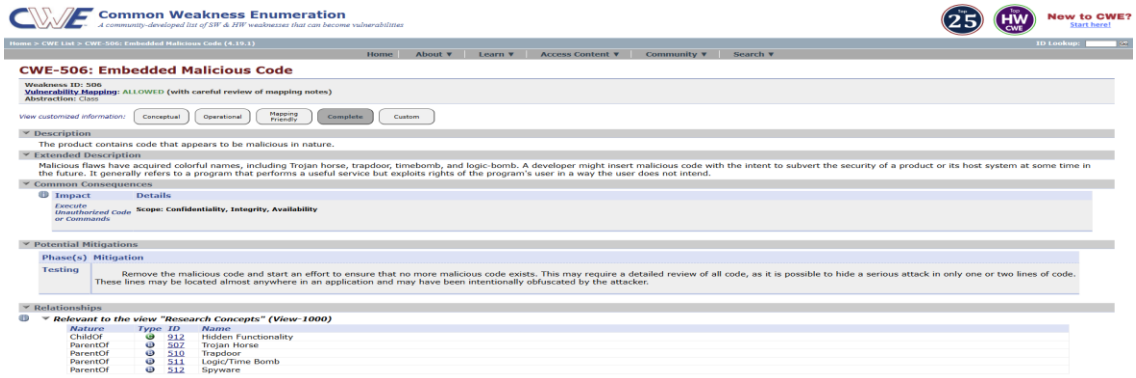
OWASP Category: A08:2021 – Software and Data Integrity Failures

Description:

This vulnerability occurs when malicious code is intentionally inserted into a software application. The code may appear to perform normal functions but secretly executes harmful actions such as backdoors, logic bombs, or Trojan-like behavior.

Business Impact:

Embedded malicious code can allow attackers to execute unauthorized commands, steal sensitive data, disrupt services, or gain persistent access to the system, affecting confidentiality, integrity, and availability.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-506: Embedded Malicious Code (4.19.1)

ID Lookup:

Home | About | Learn | Access Content | Community | Search

CWE-506: Embedded Malicious Code

Weakness ID: 506
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Class

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product contains code that appears to be malicious in nature.

Extended Description

Malicious flaws have acquired colorful names, including Trojan horse, trapdoor, timebomb, and logic-bomb. A developer might insert malicious code with the intent to subvert the security of a product or its host system at some time in the future. It generally refers to a program that performs a useful service but exploits rights of the program's user in a way the user does not intend.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	Scope: Confidentiality, Integrity, Availability

Potential Mitigations

Phase(s)	Mitigation
Testing	Remove the malicious code and start an effort to ensure that no more malicious code exists. This may require a detailed review of all code, as it is possible to hide a serious attack in only one or two lines of code. These lines may be located almost anywhere in an application and may have been intentionally obfuscated by the attacker.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf		912	Hidden Functionality
ParentOf		507	Trojan Horse
ParentOf		510	Trapdoor
ParentOf		511	Logic/Time Bomb
ParentOf		512	Spyware

Vulnerability Parameter:

Malicious or hidden code embedded within the application source code.

Steps to Reproduce:

- Review the application source code or binaries.
- Identify suspicious or hidden code segments performing unauthorized actions.
- Observe that the malicious code executes when specific conditions are triggered.

Recommendation:

- Conduct thorough code reviews and security testing.
- Use secure development practices and code analysis tools.
- Remove any malicious or suspicious code and implement strict code verification processes.

6.Vulnerability Name: Download of Code Without Integrity Check

CWE: 494 – Download of Code Without Integrity Check

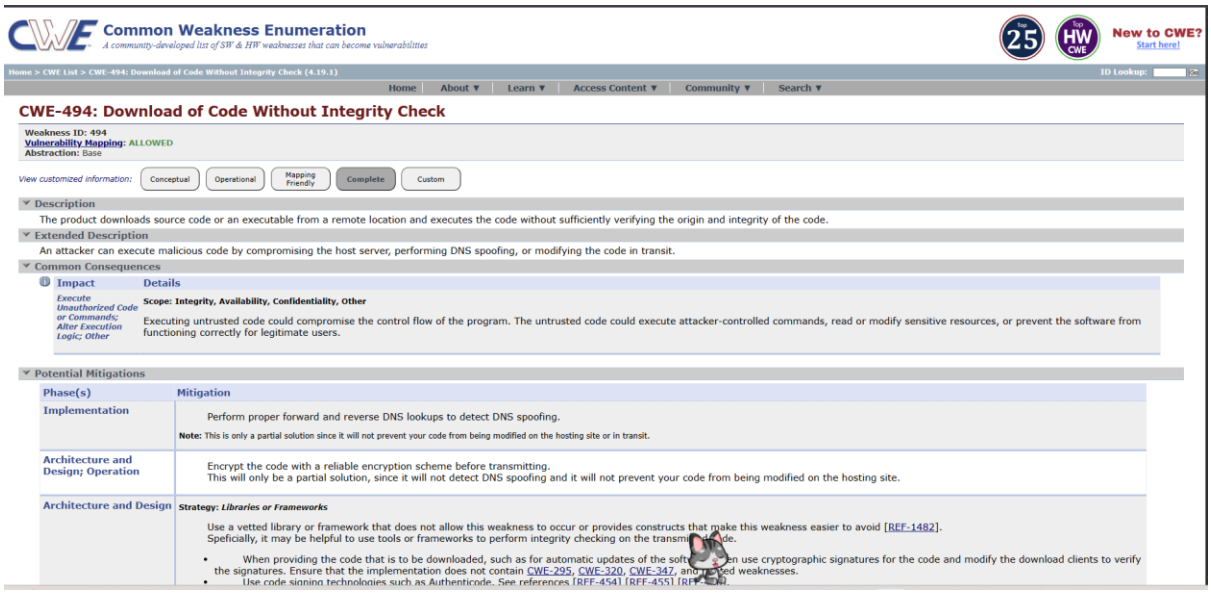
OWASP Category: A08:2021 – Software and Data Integrity Failures

Description:

This vulnerability occurs when an application downloads code or executable files from a remote source without verifying their integrity or authenticity. Attackers may modify the code during transmission or compromise the hosting server, allowing malicious code to be executed by the application.

Business Impact:

Attackers may inject malicious code that can execute unauthorized commands, steal sensitive information, or disrupt system operations. This may lead to system compromise, data breaches, and service disruption.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-494: Download of Code Without Integrity Check (4.19.1)

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-494: Download of Code Without Integrity Check

Weakness ID: 494
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product downloads source code or an executable from a remote location and executes the code without sufficiently verifying the origin and integrity of the code.

Extended Description

An attacker can execute malicious code by compromising the host server, performing DNS spoofing, or modifying the code in transit.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands; Alter Execution Logic; Other	Scope: Integrity, Availability, Confidentiality, Other Executing untrusted code could compromise the control flow of the program. The untrusted code could execute attacker-controlled commands, read or modify sensitive resources, or prevent the software from functioning correctly for legitimate users.

Potential Mitigations

Phase(s)	Mitigation
Implementation	Perform proper forward and reverse DNS lookups to detect DNS spoofing. Note: This is only a partial solution since it will not prevent your code from being modified on the hosting site or in transit.
Architecture and Design; Operation	Encrypt the code with a reliable encryption scheme before transmitting. This will only be a partial solution, since it will not detect DNS spoofing and it will not prevent your code from being modified on the hosting site.
Architecture and Design	Strategy: Libraries or Frameworks Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid [REF-1482]. Speficially, it may be helpful to use tools or frameworks to perform integrity checking on the transmitted code. • When providing the code that is to be downloaded, such as for automatic updates of the software, then use cryptographic signatures for the code and modify the download clients to verify the signatures. Ensure that the implementation does not contain CWE-295, CWE-320, CWE-347, and related weaknesses. • Use code signing technologies such as Authenticode. See references [REF-454] [REF-455] [REF-456].

Vulnerability Parameter:

Remote source URL used to download code or updates.

Steps to Reproduce:

- Identify application functionality that downloads code from an external source.
- Observe that the downloaded file is executed without verifying its integrity.
- Modify the downloaded file or intercept the connection to inject malicious code.

Recommendation:

- Verify the integrity of downloaded code using cryptographic hashes or digital signatures.
- Use secure protocols such as HTTPS when downloading files.

- Implement proper validation checks before executing downloaded code.

Humanize 182 words

7. Vulnerability Name: Weak Multi-Factor Authentication Enforcement

CWE: 308 – Use of Single Factor Authentication

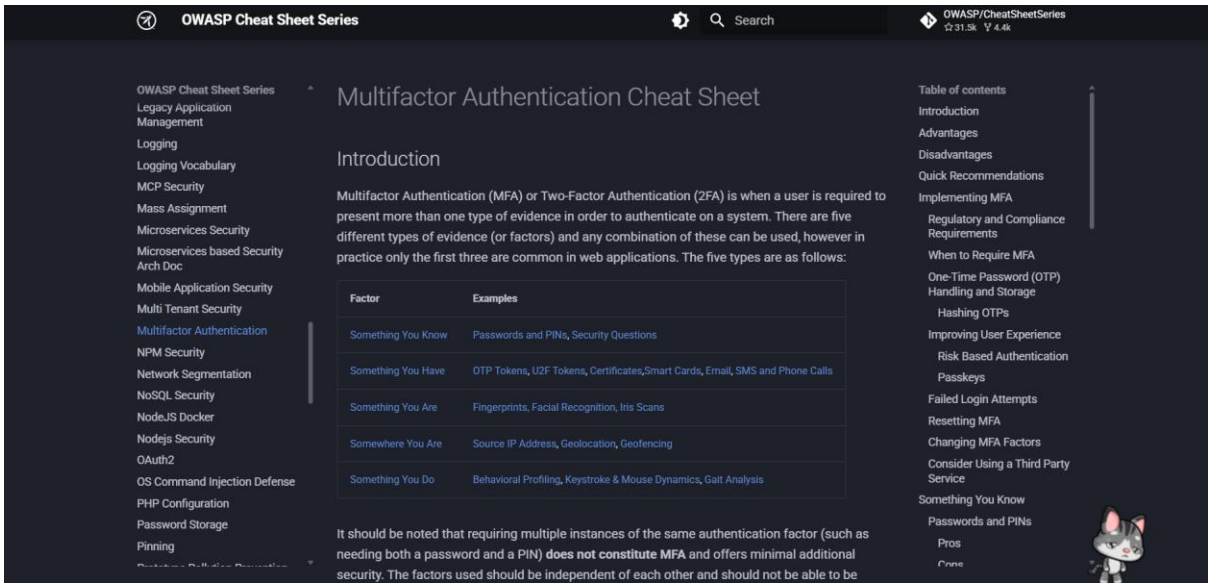
OWASP Category: A07:2021 – Identification and Authentication Failures

Description:

Improper MFA implementation allows attackers to bypass additional authentication factors.

Business Impact:

Unauthorized access to critical enterprise systems.



Vulnerability Parameter:

OTP verification process.

Steps to Reproduce:

- Intercept authentication request.
- Bypass OTP verification using replay attack.

Recommendation:

- Enforce MFA for sensitive operations.
- Use secure OTP verification methods.

8.Vulnerability Name: Weak Encoding for Password

CWE: 261 – Weak Encoding for Password

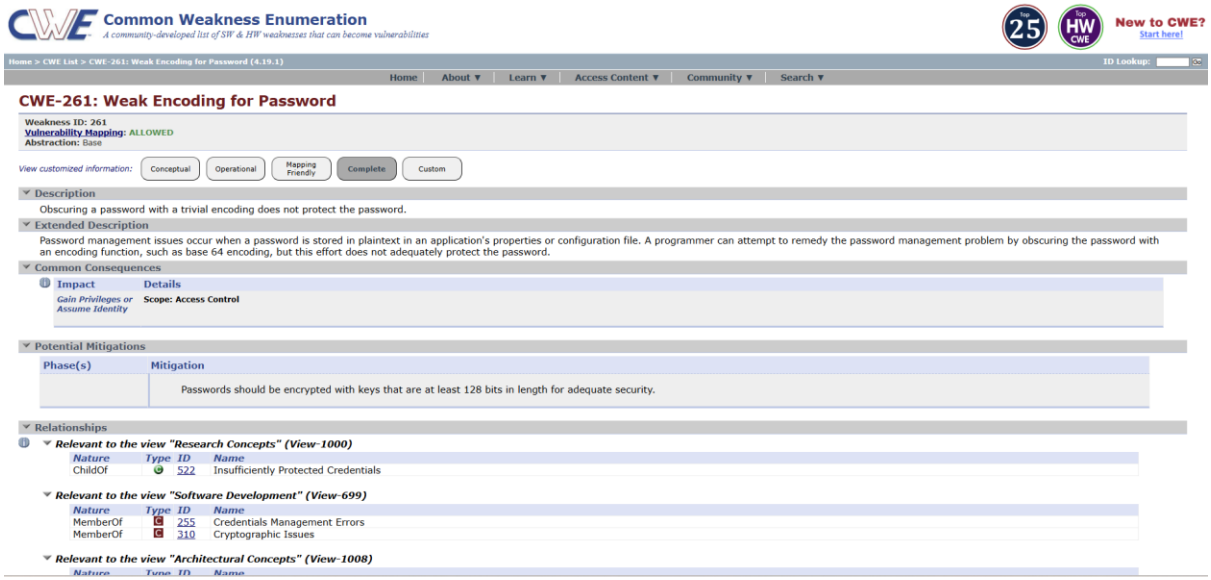
OWASP Category: A02:2021 – Cryptographic Failures

Description:

This vulnerability occurs when a password is stored using weak encoding methods such as Base64 or other simple encoding techniques instead of strong cryptographic hashing. Encoding does not protect the password because it can easily be decoded by attackers.

Business Impact:

Attackers who gain access to encoded passwords can easily decode them and obtain the original passwords. This may allow attackers to gain unauthorized access to user accounts, escalate privileges, and compromise sensitive data.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE?
Start here!

Home > CWE List > CWE-261: Weak Encoding for Password (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-261: Weak Encoding for Password

Weakness ID: 261
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Obscuring a password with a trivial encoding does not protect the password.

Extended Description

Password management issues occur when a password is stored in plaintext in an application's properties or configuration file. A programmer can attempt to remedy the password management problem by obscuring the password with an encoding function, such as base 64 encoding, but this effort does not adequately protect the password.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	Scope: Access Control

Potential Mitigations

Phase(s)	Mitigation
	Passwords should be encrypted with keys that are at least 128 bits in length for adequate security.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf		522	Insufficiently Protected Credentials

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	255	Credentials Management Errors
MemberOf	C	310	Cryptographic Issues

Relevant to the view "Architectural Concepts" (View-1008)

Vulnerability Parameter:

Weakly encoded password stored in application configuration or database.

Steps to Reproduce:

- Identify where passwords are stored in the application or configuration files.
- Observe that passwords are stored using simple encoding methods such as Base64.
- Decode the encoded value to reveal the original password.

Recommendation:

- Store passwords using strong hashing algorithms such as bcrypt, PBKDF2, or Argon2.
- Avoid storing passwords using reversible encoding methods.
- Implement proper password security and encryption standards.

9.Vulnerability Name: Missing or Improper Software Bill of Materials (SBOM) Validation

CWE: 1104 – Use of Unmaintained Third-Party Components

OWASP Category: A06:2021 – Vulnerable and Outdated Components

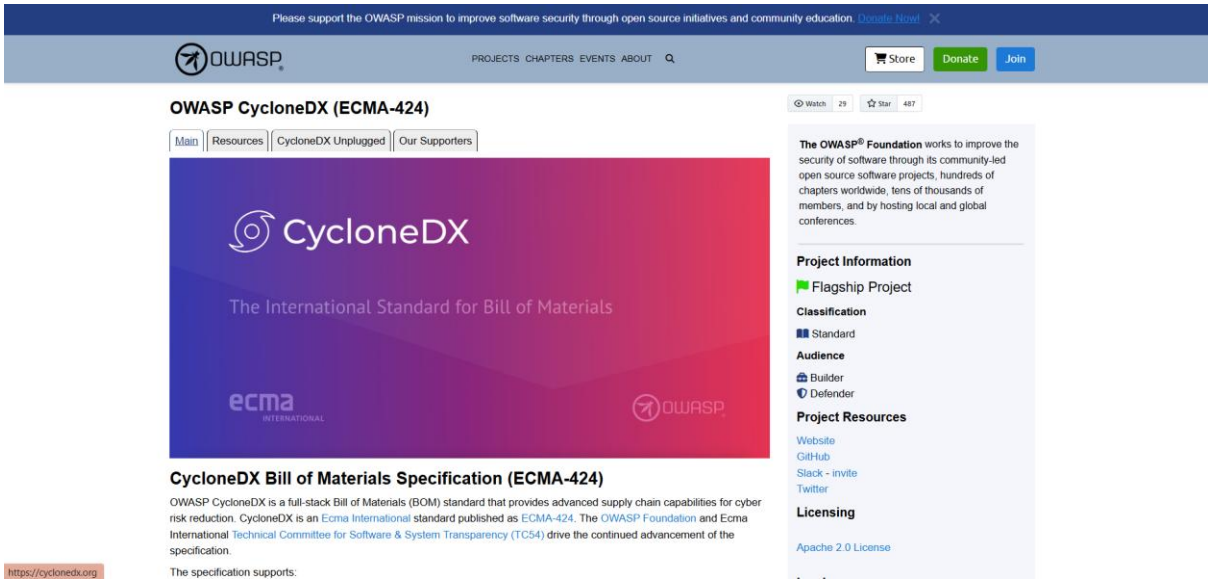
Description:

Applications often use third-party libraries and open-source components. If a proper Software

Bill of Materials (SBOM) such as **CycloneDX** is not maintained, organizations may not know which components are outdated or vulnerable. Attackers can exploit these vulnerable components to compromise the application.

Business Impact:

Exploiting vulnerable components can lead to unauthorized access, data breaches, service disruption, and financial loss. It may also cause supply-chain security risks affecting multiple systems.



Vulnerability Parameter:

Outdated third-party libraries or dependencies used in the application.

Steps to Reproduce:

- Identify third-party libraries used in the application.
- Check their versions against vulnerability databases (CVE/NVD).
- Observe if outdated or vulnerable components are still present.

Recommendation:

- Maintain a Software Bill of Materials (SBOM) using CycloneDX.
- Regularly update and patch third-party components.
- Use dependency scanning tools to detect vulnerable libraries.

10.Vulnerability Name: Missing Cryptographic Step

CWE: 325 – Missing Cryptographic Step

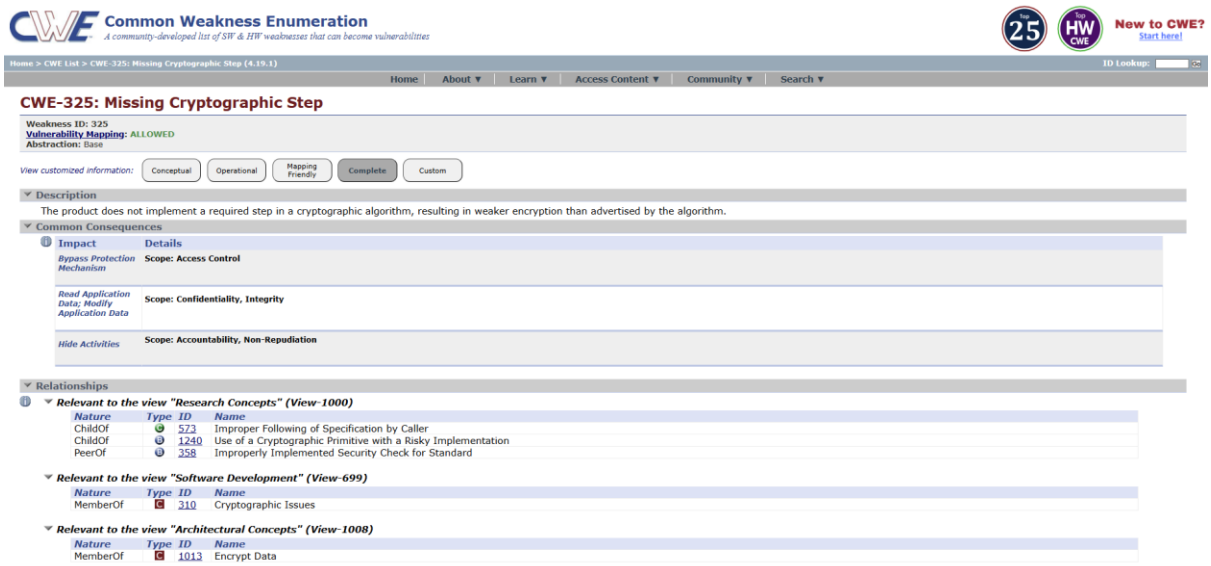
OWASP Category: A02:2021 – Cryptographic Failures

Description:

This vulnerability occurs when an application fails to implement a required step in a cryptographic process. As a result, the encryption becomes weaker than intended, making it easier for attackers to bypass security controls or access protected data.

Business Impact:

Attackers may be able to read sensitive information, modify application data, or bypass authentication mechanisms. This can lead to data breaches, loss of confidentiality, and compromise of system integrity.



Common Weakness Enumeration

A community-developed list of SW & HW weaknesses that can become vulnerabilities

New to CWE? Start here!

Home > CWE List > CWE-325: Missing Cryptographic Step (4.19.1)

ID Lookup:

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-325: Missing Cryptographic Step

Weakness ID: 325
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product does not implement a required step in a cryptographic algorithm, resulting in weaker encryption than advertised by the algorithm.

Common Consequences

Impact	Details
Bypass Protection Mechanism	Scope: Access Control
Read Application Data; Modify Application Data	Scope: Confidentiality, Integrity
Hide Activities	Scope: Accountability, Non-Repudiation

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	C	573	Improper Following of Specification by Caller
ChildOf	B	1240	Use of a Cryptographic Primitive with a Risky Implementation
PeerOf	B	358	Improperly Implemented Security Check for Standard

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	310	Cryptographic Issues

Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	C	1013	Encrypt Data

Vulnerability Parameter:

Weak or incomplete encryption process used for protecting sensitive data.

Steps to Reproduce:

- Identify the cryptographic function used in the application.

- Analyze the encryption process and check if all required steps are implemented.
- Observe that a necessary cryptographic step is missing, resulting in weak encryption.

Recommendation:

- Implement all required steps in the cryptographic algorithm.
- Use well-tested cryptographic libraries and secure standards.
- Regularly review and test encryption implementations to ensure strong security.

Stage-2

Overview:-

An overview of a **Nessus scan** typically involves several important steps and aspects that help organizations identify and manage security vulnerabilities in their systems and networks.

1. Preparation

Before starting a scan, it is important to define the objectives and scope of the vulnerability assessment. This includes identifying the systems, networks, or applications that need to be scanned. Organizations must also ensure that they have the required permissions and access rights to perform scanning on the target systems.

2. Configuration

In this stage, the scan settings are configured. A scan policy is created to define what types of vulnerabilities will be checked, how the scan will be performed, and other scanning parameters. The target assets such as IP addresses, hostnames, or network ranges are also specified.

3. Scanning

Once the configuration is completed, the Nessus scan is initiated. The tool begins scanning the selected systems using its large database of vulnerability detection plugins. It systematically analyzes the target environment to detect potential security weaknesses.

4. Vulnerability Identification

During the scan, Nessus performs a comprehensive assessment of the system. It checks for vulnerabilities such as open ports, outdated software, weak configurations, and other security issues. The identified vulnerabilities are categorized according to severity levels such as **Critical, High, Medium, Low, and Informational**.

5. Reporting

After the scanning process is completed, Nessus generates detailed reports. These reports include information about each vulnerability, its severity level, potential impact, and recommended remediation steps. Reports can be exported in different formats such as PDF, HTML, or CSV for analysis and documentation.

6. Remediation

The IT and security teams review the vulnerability reports to understand the identified issues. Based on the severity and risk level, they take corrective actions to resolve the vulnerabilities. This may involve installing security patches, updating software, modifying system configurations, or implementing stronger security controls.

7. Follow-Up

After the vulnerabilities are resolved, organizations usually perform another scan to confirm that the issues have been fixed. Regular vulnerability scanning is recommended to continuously monitor the security posture of the systems and identify new threats.

8. Compliance Auditing

Nessus can also be used to evaluate systems against various security standards and compliance frameworks. These include standards such as PCI DSS and CIS benchmarks. This helps organizations ensure that their systems meet required security and regulatory requirements.

9. Documentation

Organizations maintain records of scan results, remediation actions, and compliance checks. Proper documentation is important for security audits, reporting, and tracking improvements in the security posture.

10. User Training

To effectively use Nessus, IT and security teams should be trained in configuring scans, analyzing vulnerability reports, and implementing remediation measures. Proper training helps organizations use the tool more efficiently and improve their vulnerability management process.

It's important to note that Nessus is a valuable tool for identifying vulnerabilities, but it doesn't fix these vulnerabilities itself. It's up to the organization's IT and security teams to take the necessary steps to remediate the identified issues to enhance their security

S.No	Vulnerability Name	Severity	Nessus Plugin / Check
1	SSL Certificate Expired	Medium	SSL Certificate Expiry Check Plugin
2	SMB Signing Not Required	High	SMB Signing Configuration Plugin
3	TLS Version 1.0 Protocol Detection	Medium	TLS Protocol Detection Plugin
4	Open SSH Server Detection	Low	SSH Server Detection Plugin
5	Apache HTTP Server Outdated Version	High	Apache Web Server Version Check Plugin
6	Microsoft Windows Missing Security Updates	Critical	Windows Missing Patches Plugin
7	SSL Weak Cipher Suites Supported	Medium	SSL Cipher Suite Detection Plugin
8	Anonymous FTP Login Allowed	High	FTP Anonymous Access Plugin
9	DNS Server Cache Snooping	Medium	DNS Cache Snooping Plugin
10	SSL Self-Signed Certificate	Low	SSL Self-Signed Certificate Plugin

Target Website: testfire.net

Target IP Address: 54.83.41.36

Reports:

1. Vulnerability Name: SSL Certificate Expired

Severity: Medium

Plugin: SSL Certificate Expiry Check

Port: 443

Description:

The SSL certificate used by the server has expired. An expired certificate can no longer guarantee secure encrypted communication between the client and server.

Business Impact:

Users may experience security warnings in browsers and attackers may exploit the connection to perform man-in-the-middle attacks.

Solution:

Renew or replace the expired SSL certificate and ensure it is properly configured on the server.

2. Vulnerability Name: SMB Signing Not Required

Severity: High

Plugin: SMB Signing Configuration Check

Port: 445

Description:

The remote SMB server does not require message signing, which means communication between the client and server may not be protected from tampering.

Business Impact:

Attackers could intercept and modify SMB traffic, potentially leading to unauthorized access or data manipulation.

Solution:

Enable SMB signing in the server configuration to ensure secure communication.

3. Vulnerability Name: TLS Version 1.0 Protocol Detection

Severity: Medium

Plugin: TLS Protocol Detection Plugin

Port: 443

Description:

The server supports TLS 1.0, which is considered outdated and insecure.

Business Impact:

Attackers may exploit vulnerabilities in outdated encryption protocols to intercept sensitive data.

Solution:

Disable TLS 1.0 and enable more secure versions such as TLS 1.2 or TLS 1.3.

4. Vulnerability Name: Open SSH Server Detection

Severity: Low

Plugin: SSH Server Detection Plugin

Port: 22

Description:

An SSH service is running on the target system. If not properly secured, it may allow unauthorized remote access attempts.

Business Impact:

Attackers may attempt brute-force attacks on the SSH service to gain access to the system.

Solution:

Use strong passwords, enable key-based authentication, and restrict SSH access to trusted IP addresses.

5. Vulnerability Name: Apache HTTP Server Outdated Version

Severity: High

Plugin: Apache Version Detection Plugin

Port: 80

Description:

The Apache web server running on the system is outdated and may contain known vulnerabilities.

Business Impact:

Attackers may exploit known vulnerabilities in outdated web server versions to gain unauthorized access or execute malicious code.

Solution:

Update the Apache server to the latest stable version and apply security patches.

6. Vulnerability Name: Windows Missing Security Updates

Severity: Critical

Plugin: Windows Missing Patches Plugin

Port: 445

Description:

The system is missing important Microsoft security updates and patches.

Business Impact:

Attackers may exploit known vulnerabilities in unpatched systems to gain full control over the system.

Solution:

Install the latest security updates from Microsoft and enable automatic updates.

7. Vulnerability Name: SSL Weak Cipher Suites Supported

Severity: Medium

Plugin: SSL Cipher Suite Detection Plugin

Port: 443

Description:

The server supports weak encryption cipher suites that may be vulnerable to cryptographic attacks.

Business Impact:

Sensitive data transmitted between users and the server could be intercepted or decrypted by attackers.

Solution:

Disable weak cipher suites and enable only strong encryption algorithms.

8. Vulnerability Name: Anonymous FTP Login Allowed

Severity: High

Plugin: FTP Anonymous Access Plugin

Port: 21

Description:

The FTP server allows anonymous login without authentication.

Business Impact:

Attackers may upload, download, or modify files on the server without authorization.

Solution:

Disable anonymous FTP access and enforce authentication.

9. Vulnerability Name: DNS Server Cache Snooping

Severity: Medium

Plugin: DNS Cache Snooping Plugin

Port: 53

Description:

The DNS server allows external users to query its cache.

Business Impact:

Attackers may gather information about internal network activity and domain queries.

Solution:

Configure the DNS server to restrict cache queries from external networks.

10. Vulnerability Name: SSL Self-Signed Certificate

Severity: Low

Plugin: SSL Self-Signed Certificate Plugin

Port: 443

Description:

The server uses a self-signed SSL certificate instead of one issued by a trusted certificate authority.

Business Impact:

Users may receive security warnings and attackers may exploit the lack of trust to perform spoofing attacks.

Solution:

Replace the self-signed certificate with one issued by a trusted certificate authority.

Stage – 3

Title

Design and apply context-aware access controls based on user identity, device posture, location, and application risk.

Context-aware access control is an advanced security approach used in modern Zero Trust architectures. Instead of granting access based only on usernames and passwords, it evaluates multiple contextual factors before allowing users to access applications and systems. These factors include user identity, device posture, location, and the risk level of the application being accessed. This approach ensures that access decisions are dynamic and adaptive based on the security context of each request.

The first layer of context-aware access control is **user identity**. Authentication is performed through an Identity Provider such as Azure Active Directory or Okta, often using secure authentication protocols and multi-factor authentication (MFA). This ensures that only verified users are allowed to request access. Access policies can then be defined based on user roles, departments, or group memberships within the organization.

The second layer is **device posture**, which evaluates the security condition of the device attempting to access the system. Security tools such as Endpoint Detection and Response (EDR) solutions or Mobile Device Management (MDM) platforms can verify whether the device meets security requirements. These checks may include verifying whether antivirus software is active, the operating system is updated, disk encryption is enabled, and no suspicious activities are detected. If a device fails these checks, access can be restricted or limited to remediation services.

The third layer is **location awareness**, which analyzes the geographic or network location from which the access request originates. Access attempts from trusted corporate networks or approved IP ranges may be allowed, while requests from suspicious locations or anonymous VPN services may require additional authentication or be blocked entirely. This helps reduce the risk of unauthorized access from unknown or high-risk regions.

The final layer is **application risk assessment**. Different applications have different levels of sensitivity, and access policies must reflect this. For example, internal informational websites may require basic authentication, while critical systems such as financial platforms or

customer databases require stricter security controls. By categorizing applications based on risk levels, organizations can enforce stronger policies for high-value assets.

By combining these contextual signals, organizations can implement adaptive security policies. Access decisions are made dynamically based on the user, device, location, and application risk. This ensures that security controls are proportional to the risk level of each access attempt while maintaining usability for legitimate users.

Future Scope

Stage 1: Future Scope of Web Application Testing

The future scope of web application testing is continuously expanding due to the rapid growth of digital services and online platforms. As organizations increasingly rely on web applications for business operations, the importance of thorough testing and security assessment will continue to grow.

Some key aspects of its future scope include:

▪ **Increased Demand:** With the expansion of online services and e-commerce platforms, the demand for secure and reliable web applications will continue to rise. This will significantly increase the need for web application testing to ensure functionality, performance, and security.

▪ **Advanced Technologies:** The integration of technologies such as Artificial Intelligence (AI), Internet of Things (IoT), blockchain, and cloud computing is making web applications more complex. Testing these advanced systems will require more sophisticated tools and specialized skills.

▪ **Security Emphasis:** Cyber threats and data breaches are increasing rapidly. As a result, security testing will become a critical component of web application development. Testers will need to focus on vulnerability detection, secure coding practices, and compliance with standards such as OWASP guidelines and GDPR regulations.

▪ **Mobile and Cross-Browser Testing:** With the growing use of smartphones and tablets, web applications must work smoothly across multiple devices and browsers. Cross-platform compatibility testing will become an essential part of web application testing.

▪ **Performance and Load Testing:** As user expectations for speed and responsiveness increase, performance testing techniques such as load testing, stress testing, and scalability testing will become even more important.

Stage 2: Future Scope of Vulnerability Assessment and Testing

The future of vulnerability assessment and security testing is highly promising due to the continuous evolution of cyber threats and the increasing importance of proactive security strategies.

Key future developments include:

▪ **AI-Driven Vulnerability Detection:** Artificial Intelligence and Machine Learning will significantly improve vulnerability scanning tools by automating threat detection and identifying complex security flaws that may be missed during manual testing.

▪ **IoT and Mobile Security Testing:** With the rapid adoption of IoT devices and mobile applications, security testing will expand to cover connected devices, mobile platforms, and embedded systems.

▪ **Blockchain and Cryptocurrency Security:** As blockchain technology and digital assets continue to grow, security testing will be required to ensure the protection of blockchain networks, smart contracts, and cryptocurrency platforms.

▪ **Serverless Security Testing:** The adoption of serverless architectures and cloud-native applications will require new testing approaches to ensure the security of APIs, microservices, and serverless functions.

▪ **Container and Cloud Security:** Technologies such as Docker and Kubernetes are widely used for deploying applications. Security testing will increasingly focus on container images, orchestration configurations, and cloud workloads.

Stage 3: Future Scope of Context-Aware Access Control

The implementation of context-aware access control based on Zero Trust principles opens the door for more advanced and adaptive security mechanisms in the future.

Future improvements may include:

▪ **Integration with Secure Access Service Edge (SASE):** Combining private application access with internet security platforms to create a unified cloud security architecture.

▪ **Advanced Device Posture Assessment:** Using endpoint detection and response (EDR) tools to evaluate real-time device risk scores and automatically block access from compromised devices.

▪ **Privileged Access Management:** Providing secure and monitored access for third-party vendors, contractors, and administrators with time-limited permissions.

▪ **IoT and OT Security Segmentation:** Applying context-aware access control to Internet of Things (IoT) and Operational Technology (OT) devices to prevent unauthorized communication within networks.

▪ **AI-Based Anomaly Detection:** Using machine learning to analyze user behavior and detect unusual access patterns such as abnormal login locations or unusual access times.

▪ **Cloud Workload Protection:** Securing communication between microservices and containers across multi-cloud environments such as AWS, Azure, and Google Cloud.

▪ **Automated Access Management:** Automatically granting or revoking access permissions when employees join, leave, or change roles within the organization.

▪ **Digital Experience Monitoring:** Monitoring application performance and user experience to detect network issues and ensure smooth connectivity.

Topics Explored

1. Zscaler Security Platform
2. Zero Trust Security Architecture
3. Zscaler Private Access (ZPA)
4. Virtual Private Networks (VPN)
5. Malware Information Sharing Platform (MISP)
6. College Network Infrastructure Analysis
7. Deployment of Security Operations Center (SOC) in the College

8. Threat Intelligence and Cyber Threat Monitoring
9. Incident Response and Security Management
10. Web Application Security Testing (OWASP)

Tools Explored

▪ **Nessus Vulnerability Scanner** – Used for identifying vulnerabilities, misconfigurations, and security weaknesses in systems and web applications.

▪ **OWASP Testing Techniques** – Used for identifying common web application vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and authentication weaknesses.

▪ **Security Monitoring Tools** – Used for analyzing network activity and detecting potential security threats.